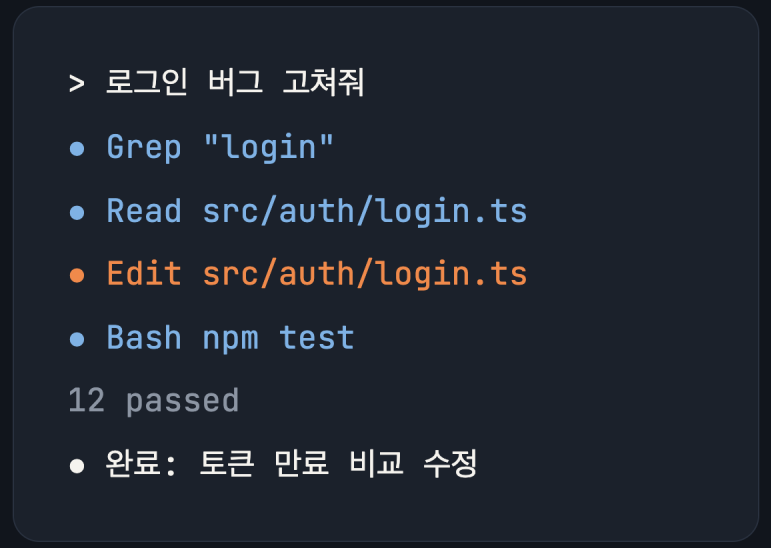
LECTURE · AI CODING AGENTS

AI 코딩 에이전트는 어떻게 일하는가

내부 아키텍처와 Agent Loop부터
MCP, 멀티 에이전트, 엔터프라이즈 운영까지

Claude Code · Codex · Cursor



COURSE GOAL

이 강의의 목표

AI 코딩 도구를 '쓰는' 수준을 넘어, 동작 원리를 이해하고
직접 **설계 · 확장 · 운영**하는 엔지니어링 역량을 기릅니다.

01

이해

내부 아키텍처, Agent Loop, Tool Calling, Context Engineering

02

설계 · 확장

MCP 연동, 멀티 에이전트 구조, LLM 추론 성능 최적화

03

운영

보안, 코드 품질 평가, 장애 분석, Observability

강의 교재를 배부합니다

ROADMAP

강의를 이끄는 여섯 가지 질문

Q1

명령을 입력하면 내부에서 어떤 일이 일어나는가?

→ Agent Loop

Q2

수많은 파일 중 필요한 코드를 어떻게 찾는가?

→ Context Engineering

Q3

도구를 어떻게 선택·실행하고 외부와 연결하는가?

→ Tool Calling · MCP

Q4

복잡한 작업을 어떻게 나누고 협력하는가?

→ 멀티 에이전트

Q5

잘못된 코드와 반복 오류의 원인을 어떻게 찾는가?

→ 성능 최적화 · 장애 분석

Q6

품질과 보안을 어떻게 검증하고 운영하는가?

→ 품질 · 보안 · Observability

01

Agent Loop

명령 한 줄이 코드 변경이 되기까지

ANATOMY

에이전트 = 모델 + 도구 + 루프

모델 (LLM)

요청과 현재 맥락을 읽고 다음에 할 행동을 결정합니다

+

도구 (Tools)

파일 읽기·쓰기, 코드 검색, 터미널 명령, 외부 시스템

+

하네스 (루프)

도구를 실행하고 결과를 돌려주며 권한과 컨텍스트를 관리합니다

모델은 '무엇을 할지'를 텍스트로 출력할 뿐, 실제 실행은 하네스가 합니다.

THE LOOP

수집 → 행동 → 검증 → 반복

1
컨텍스트 수집
파일 검색·읽기, 지침과 이력 확인

→

2
행동
도구 호출로 코드 수정, 명령 실행

→

3
검증
테스트·빌드·린트 결과를 관찰

→

4
판단
목표 달성 여부와 다음 행동 결정

계속
도구 결과를 대화에 붙이고 모델을 다시 호출 → 1로 돌아감

종료
도구 호출 없이 최종 답변을 내거나, 턴·예산 한도에 도달

UNDER THE HOOD

루프의 실체는 단순한 반복문

```
messages = [system_prompt, user_request]
while True:
    response = llm(messages, tools)
    messages.append(response)
    if not response.tool_calls:
        break  # 최종 답변
    for call in response.tool_calls:
        result = run_tool(call)  # 권한 확인 후 실행
        messages.append(result)
```

상태는 messages

모든 도구 결과가 대화 이력에 누적됩니다

모델은 무상태

매 호출마다 전체 이력을 다시 읽습니다

종료는 모델이 결정

도구 호출 없는 응답이 나오면 끝납니다

ONE REQUEST, MANY CALLS

요청은 한 번, 모델 호출은 여러 번

예시 요청: **“로그인 실패 버그 고쳐줘”**

턴	모델의 출력	하네스의 동작
1	Grep "login" 호출	일치하는 파일 목록 반환
2	Read src/auth/login.ts 호출	파일 내용 반환
3	Edit: 만료 시간 비교 수정	파일에 변경 적용
4	Bash: npm test 호출	테스트 결과(통과) 반환
5	변경 요약 (도구 호출 없음)	루프 종료, 사용자에게 표시

02

Context Engineering

모델은 컨텍스트에 있는 것만 안다

CODE RETRIEVAL

필요한 코드를 찾는 세 가지 방식

AGENTIC SEARCH

도구로 직접 탐색

grep·glob으로 찾고 파일을 읽으며 범위를 좁혀 감

장점 항상 최신, 인덱스 불필요
한계 탐색에 턴과 토큰 소요

예: Claude Code, Codex CLI

EMBEDDING INDEX

의미 기반 검색

코드를 조각내 임베딩하고 질문과 비슷한 코드를 검색

장점 이름을 몰라도 의미로 찾음
한계 인덱스 구축·갱신 필요

예: Cursor 코드베이스 인덱싱

CODE STRUCTURE

구조 정보 활용

LSP, 심볼 그래프, 저장소 맵으로 정의·참조 추적

장점 정의·참조를 정확히 추적
한계 언어별 도구 지원 필요

예: Aider repo map, LSP 연동

실제 에이전트는 이 방식들을 **조합**해서 씁니다.

CONTEXT WINDOW

컨텍스트 창에 들어가는 것들

시스템 프롬프트 · 도구 정의	고정
프로젝트 지침 (CLAUDE.md 등)	고정
대화 이력	누적
도구 결과: 파일 내용 · 명령 출력	빠르게 증가
현재 요청	새 입력

매 턴, 모델에게 무엇을 보여주고 무엇을 뺄지 설계하는 일

컨텍스트는 유한합니다. 길어질수록 비용과 지연이 늘고, 중요한 정보가 묻힐 수 있습니다.

프로젝트 지침 파일: CLAUDE.md (Claude Code), AGENTS.md (Codex), .cursor/rules (Cursor)

TECHNIQUES

긴 작업을 위한 컨텍스트 관리 기법

COMPACTION

대화 압축

컨텍스트가 차면 이전 이력을 요약본으로 바꿔 작업을 이어갑니다

JUST-IN-TIME

필요할 때만 읽기

파일 전체 대신 경로·검색 결과로 시작해 필요한 부분만 읽습니다

ISOLATION

서브에이전트 격리

탐색은 별도 컨텍스트에서 하고 요약만 주 에이전트에 돌려줍니다

MEMORY

외부 메모리

진행 상황과 결정 사항을 파일에 기록해 두고 다시 읽습니다

03

Tool Calling & MCP

모델이 바깥 세상에 손을 뻗는 방법

TOOL CALLING

Tool Calling은 이렇게 동작합니다

1 정의 전달

이름·설명·JSON Schema를 모델에 제공

→

2 호출 생성

모델이 도구 이름과 인자를 구조화해 출력

→

3 실행

하네스가 권한을 확인하고 실제로 실행

→

4 결과 반환

결과를 tool_result로 대화에 추가

도구 정의 (요청에 포함)

```
{"name": "run_command",
 "description": "터미널 명령 실행",
 "input_schema": {"type": "object",
  "properties": {"command": {"type": "string"}}}}
```

모델의 출력 (tool_use)

```
{"type": "tool_use",
 "name": "run_command",
 "input": {"command": "npm test"}}
```

TOOL DESIGN

좋은 도구의 다섯 가지 조건

원칙	이유
명확한 이름과 설명	모델은 설명을 읽고 도구를 고릅니다. 설명이 곧 프롬프트입니다
적고 겹치지 않는 도구	비슷한 도구가 많으면 선택이 흔들리고 컨텍스트가 낭비됩니다
토큰 효율적인 출력	긴 결과는 잘라내거나 페이지 단위로, 필요한 정보만 돌려줍니다
행동 가능한 오류	무엇이 틀렸고 어떻게 고칠지 알려야 모델이 스스로 복구합니다
읽기와 쓰기의 분리	위험도에 따라 서로 다른 권한 정책을 적용할 수 있습니다

PERMISSIONS & SANDBOX

실행 전에 확인하고, 격리해서 실행한다

ALLOW

자동 허용

파일 읽기, 코드 검색, 목록 조회

부작용이 없는 작업

ASK

승인 후 실행

파일 수정, 셸 명령, 커밋·푸시

되돌리기 어려운 작업은 사람이 확인

DENY

차단 · 격리

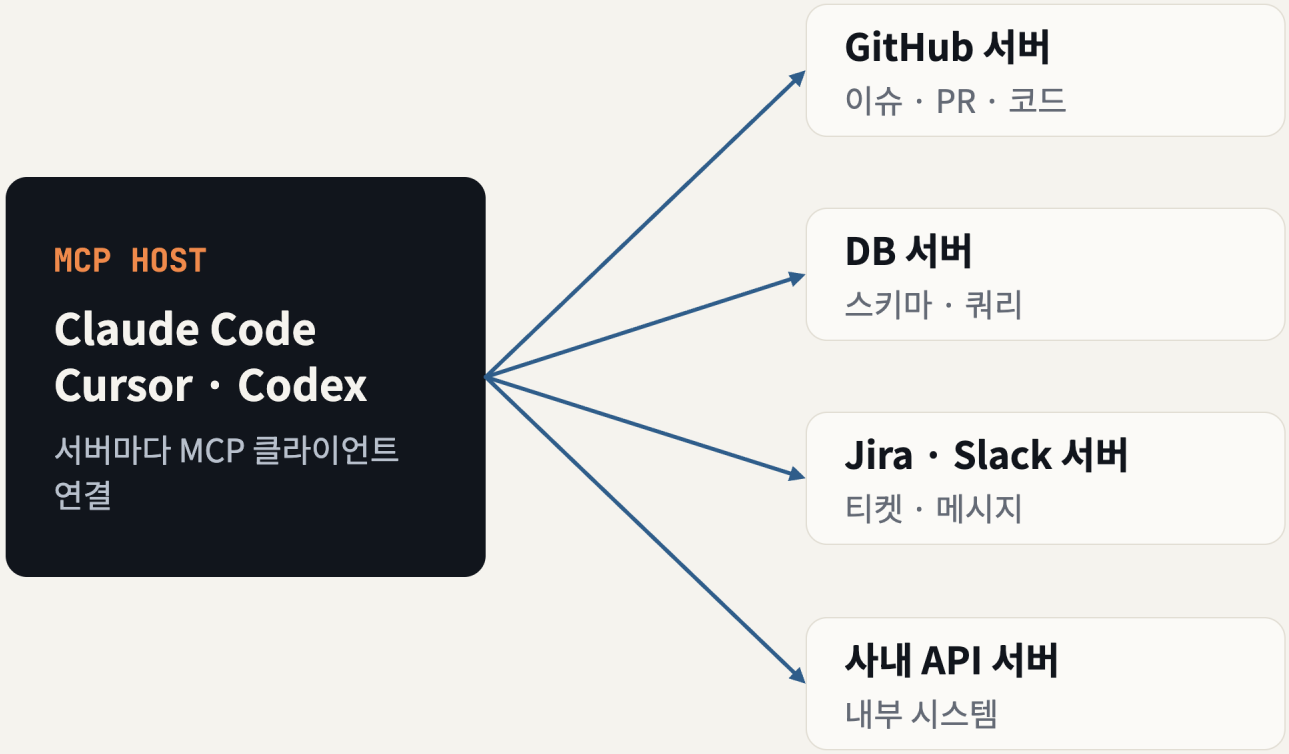
시크릿 파일, 프로젝트 밖 경로, 허용되지 않은 네트워크

정책으로 막고 샌드박스로 한 번 더

규칙은 사용자 · 프로젝트 · 조직 단위 설정으로 관리하고, 실행은 컨테이너 같은 샌드박스로 파일 시스템과 네트워크를 제한합니다.

MODEL CONTEXT PROTOCOL

MCP: 외부 시스템 연결의 표준



N×M 개별 연동 대신
하나의 프로토콜

서버를 한 번 만들면 여러 에이전트에서 재사용합니다

Tools 모델이 호출하는 기능
Resources 읽을 수 있는 데이터
Prompts 재사용 프롬프트

JSON-RPC 기반 · 전송: stdio(로컬), Streamable HTTP(원격)

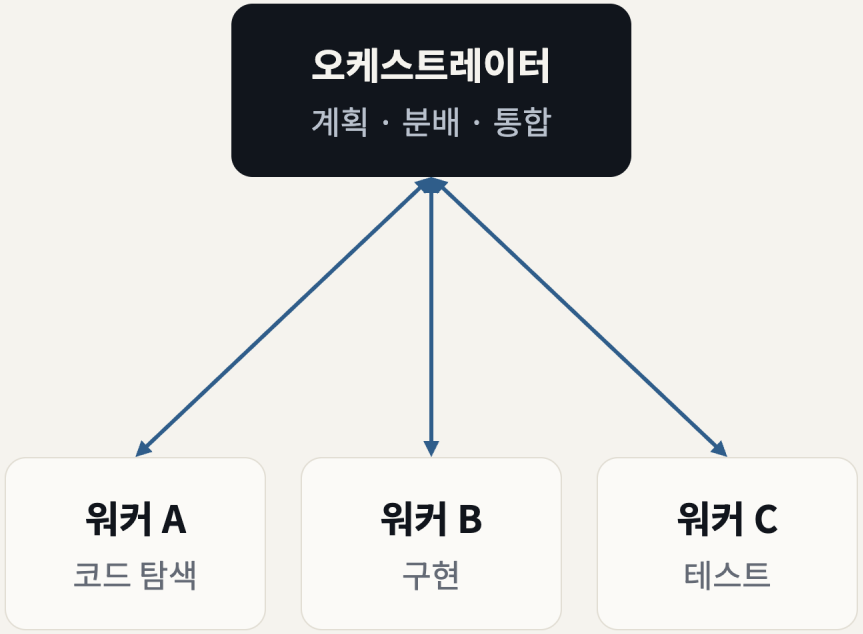
04

멀티 에이전트 설계

나누어 맡기고, 결과를 모은다

PATTERNS

작업을 나누는 세 가지 패턴



작업 지시는 아래로, 요약된 결과는 위로

오케스트레이터-워커

주 에이전트가 계획하고, 하위 작업을 서브에이전트에 맡긴 뒤 결과를 통합

병렬 실행

독립된 작업을 git worktree 같은 분리된 작업 공간에서 동시에 진행

역할 분리

작성 · 리뷰 · 테스트 에이전트가 서로의 결과를 검증

WHEN TO SPLIT

멀티 에이전트, 언제 쓸까

적합한 경우

- 넓게 탐색·조사해야 하는 작업
- 서로 독립된 모듈·파일 단위 작업
- 컨텍스트를 나눠야 하는 대규모 작업
- 독립적인 리뷰·검증이 필요한 작업

피해야 할 경우

- 단계가 강하게 얽힌 순차 작업
- 여러 에이전트가 같은 파일을 동시 수정
- 단일 에이전트로 충분한 작은 작업

에이전트가 늘수록 토큰 비용과 조정 비용도 늘어납니다. 이득이 비용보다 클 때만 나눕니다.

05

성능 최적화와 장애 분석

더 빠르고 싸게, 그리고 왜 실패했는지 알기

INFERENCE OPTIMIZATION

LLM 추론 성능 최적화

작업 시간 ≈ 모델 호출 횟수 × (입력 처리 + 출력 생성) + 도구 실행 시간

기법	원리	적용 포인트
프롬프트 캐싱	반복되는 앞부분의 계산 결과를 재사용	변하지 않는 내용을 앞에 고정
모델 라우팅	작업 난이도에 맞는 모델 선택	탐색·요약은 경량 모델
병렬 도구 호출	독립된 호출을 한 턴에 동시 실행	턴 수와 대기 시간 감소
컨텍스트 축소	입력 토큰을 줄여 비용·지연 감소	긴 출력 요약·잘라내기
스트리밍	생성되는 대로 바로 표시	체감 대기 시간 감소

FAILURE MODES

에이전트가 실패하는 대표 패턴

증상	흔한 원인	대응
없는 API를 사용	코드를 확인하지 않고 추측	정의를 먼저 읽게 하고 타입체크로 검증
같은 오류를 반복	실패한 시도가 컨텍스트에 쌓여 판단을 흐림	원인을 기록한 뒤 새 컨텍스트에서 재시작
엉뚱한 파일 수정	탐색 부족, 모호한 요청	범위를 명시하고 계획을 먼저 검토
테스트만 통과시키는 수정	검증 기준이 약하거나 테스트 수정을 허용	테스트 변경 금지 규칙, 리뷰 단계 추가
끝나지 않는 루프	종료 조건과 한도가 없음	최대 턴 수·비용 한도 설정

ROOT CAUSE ANALYSIS

원인을 찾는 다섯 단계

모델을 탓하기 전에, 모델이 무엇을 보았는지부터 봅니다.

1
트레이스 확인
무엇을 읽고 무엇을 호출했나

2
컨텍스트 점검
필요한 정보가 있었나, 너무 많지 않았나

3
도구 결과 확인
오류가 잘리거나 숨겨지지 않았나

4
지침 보완
규칙을 CLAUDE.md · AGENTS.md에 추가

5
회귀 평가
실패 사례를 평가셋에 추가

06

품질 · 보안 · 운영

엔터프라이즈 환경에서 안전하게 운영하기

CODE QUALITY

생성된 코드의 품질 검증

LAYER 1 **결정적 검증**	빌드 · 타입체크 · 린트 · 테스트. 가장 싸고 확실한 피드백으로, 에이전트가 루프 안에서 직접 실행합니다
LAYER 2 **리뷰**	사람 리뷰와 AI 리뷰 에이전트가 설계, 가독성, 요구사항 충족 여부를 판단합니다
LAYER 3 **평가셋**	공개 벤치마크(SWE-bench 등)와 사내 과제로 모델·프롬프트·도구 변경 시 회귀를 측정합니다

추적할 지표 작업 성공률 · 테스트 통과율 · 리뷰 후 수정 비율 · 배포 후 결함

SECURITY

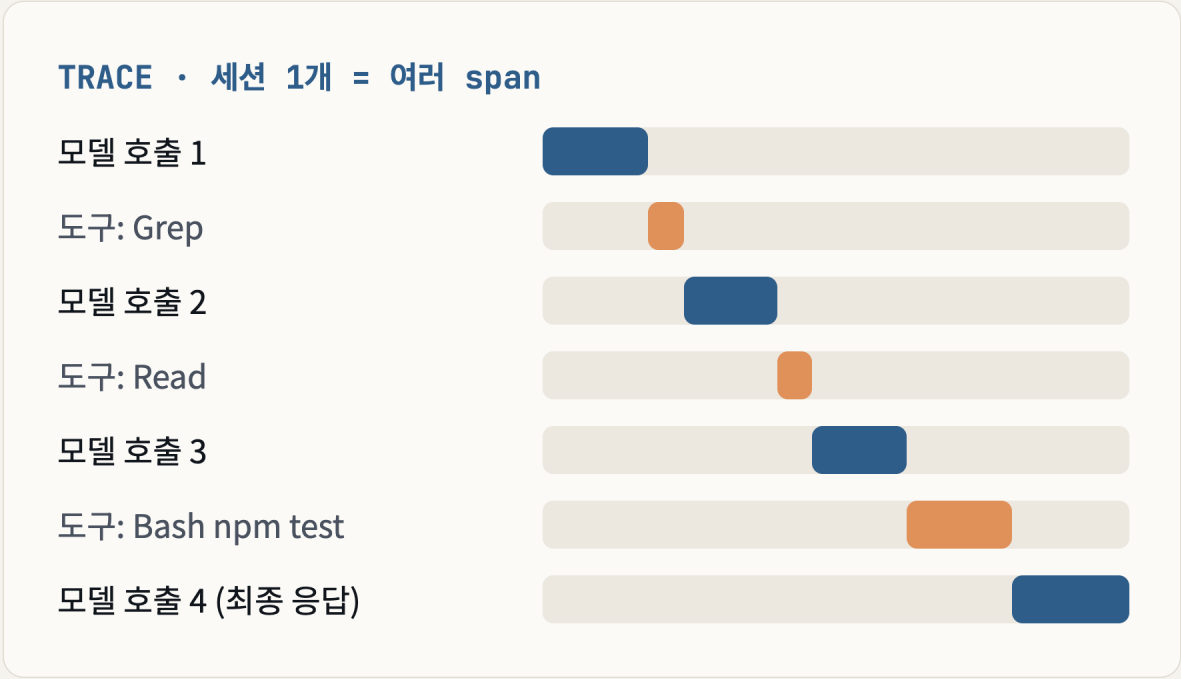
AI 에이전트의 보안 위협과 방어

위협	예시	방어
프롬프트 인젝션	README·이슈·웹 페이지에 숨긴 지시를 따름	외부 콘텐츠는 데이터로 취급, 고위험 작업은 사람 승인
시크릿 유출	.env나 토큰을 읽어 외부로 전송	시크릿 경로 차단, 네트워크 허용 목록
위험한 명령	파일 일괄 삭제, 강제 push, 운영 DB 변경	권한 정책, 샌드박스 실행
공급망 공격	존재하지 않거나 악성인 패키지 설치	의존성 검토, 사내 레지스트리
취약한 코드	SQL 인젝션, 인증·권한 검사 누락	정적 분석(SAST), 보안 리뷰

원칙 최소 권한 · 다층 방어 · 사람의 최종 승인

OBSERVABILITY

에이전트를 관찰 가능하게



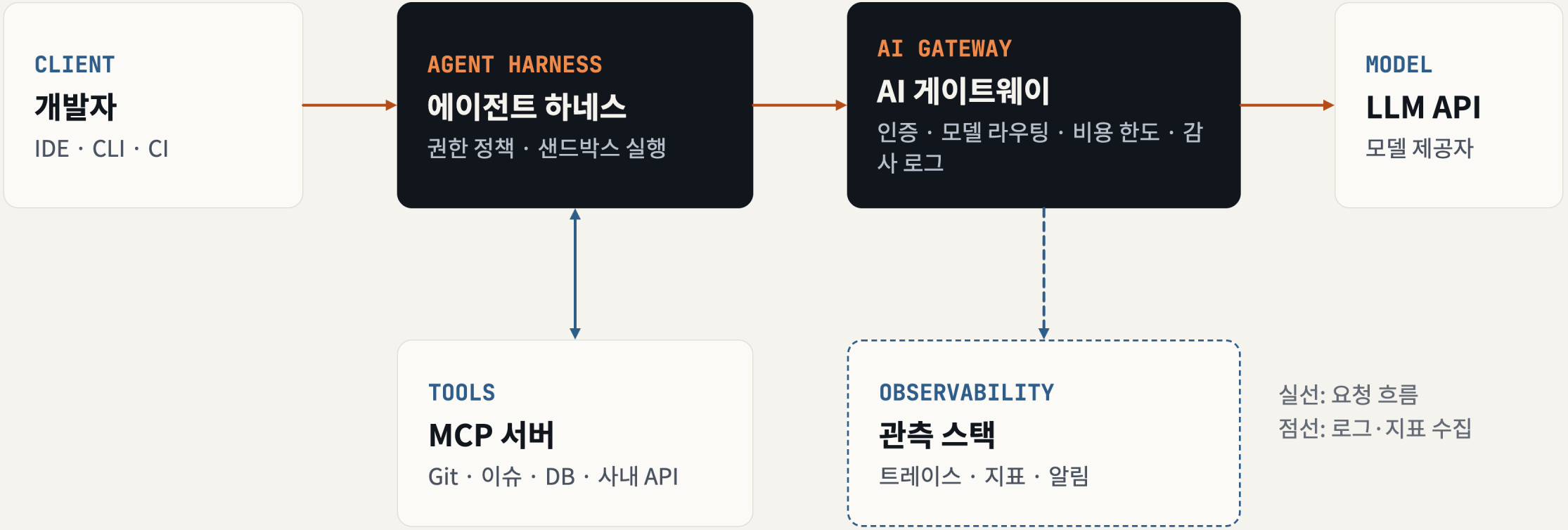
핵심 지표

- 작업 성공률
- 요청당 토큰 · 비용
- 지연 시간 (p50 / p95)
- 도구 오류율 · 재시도 횟수
- 사람 개입 횟수

OpenTelemetry 같은 표준으로 수집해 기존 모니터링 스택에 연결합니다

REFERENCE ARCHITECTURE

엔터프라이즈 운영 아키텍처



WRAP-UP

여섯 가지 질문에 대한 답

Q1 · AGENT LOOP

모델이 도구 호출을 출력하고 하네스가 실행해 돌려주는 반복

Q2 · CONTEXT

탐색·인덱스·구조 정보로 찾고, 컨텍스트에 넣을 것을 설계

Q3 · TOOLS & MCP

스키마로 정의한 도구를 권한 아래 실행, MCP로 표준 연결

Q4 · MULTI-AGENT

독립적인 작업만 나누고, 요약된 결과를 통합

Q5 · DEBUGGING

트레이스로 모델이 본 것을 확인하고, 지침·평가셋으로 재발 방지

Q6 · OPERATIONS

결정적 검증 + 리뷰 + 최소 권한 + Observability

Q & A

질문과 토론

AI 코딩 도구를 쓰는 사람에서,
설계하고 운영하는 엔지니어로